

PokéAPI - Parallel and Distributed Systems

Levy Silva¹

¹Instituto Federal de Pernambuco - IFPE
Campus Belo Jardim

levy.silva@belojardim.ifpe.edu.br

Abstract. *This report describes the project assignment for Parallel and Distributed Systems course. In short, the goal is to perform parallel requests to PokéAPI to retrieve Pokémon data. The work must be done individually, and the code must be submitted to your GitHub. The deadline will be announced soon.*

1. PokéAPI

The PokéAPI¹ is a free open-source RESTful API for collecting Pokémon data (abilities, game indices, images, species and so on), serving over 50 billion API calls each month. The <https://pokeapi.co/api/v2/pokemon> endpoint is the core of the API, enabling to get data by pokémon names, sequential ids or query arguments. Specifically, it enables requests to be made as several types including, but not limited to: (i) pokémon name (<https://pokeapi.co/api/v2/pokemon/bulbasaur>); (ii) pokémon id (<https://pokeapi.co/api/v2/pokemon/1>); and (iii) query args, for example, by using *limit* argument (<https://pokeapi.co/api/v2/pokemon?limit=10>). For the third case, the return is a tuple of pokémon: (*name*, *url*). Use such URL (sequential format) to also retrieve the data about the pokémon (as e.g., for *bulbasaur* <https://pokeapi.co/api/v2/pokemon/1>). Also, the JSON format is used on the API response, and there is no authentication step.

Toward this front, the goal of this project is to perform parallel requests to collect a subset of pokémon images from this API, also storing each one on the disk. Specifically, given the API response, we aim to obtain the key **front_default** (inside *sprites*), which includes the main image URL for the pokémon. Note that the **front_default** stores the image URL, which will need to be downloaded to the disk. In addition, our goal is to compare the parallel download time against a sequential approach, over the same number of pokémon requests. Finally, implementation details are described next.

2. Requirements

In order to execute this project, consider the following requirements: (1) Use python (or your preferred programming language) on this project; (2) Design the sequential methodology to get pokémon images; (3) Implement a parallel approach using the python libs: **multiprocessing**, **threading** and **concurrent.futures**. You should implement it for all 3 approaches; (4) Apply python **requests** to call the API; (5) Utilize python **os** to store the images on disk; (6) Compare the final time to get 100, 500 and 1,000 pokémon images; (7) Run each approach 10 (times) to get the average execution time; (8) Use distinct setups of threads/processes (2, 4, or 8 as your CPU availability);

Finally, write a report using LaTeX and the SBC template. The documentation needs to include at least: context, results (tables and/or charts) and discussion.

¹<https://pokeapi.co>